20. To Capture SIGTSTP, Suspend Processes, Update Job Table, Preserve State, Resume Later, Test Suspension Workflow.

Code:

```
  GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>
#include <string.h>

#define MAX_JOBS 5

typedef enum {
    RUNNING,
    STOPPED,
    COMPLETED
} JobState;

typedef struct {
    int job_id;
    pid_t pid;
    char command[100];
    JobState state;
} Job;

Job jobTable[MAX_JOBS];
int jobCount = 0;

pid_t foregroundPid = -1;

/* Handle Ctrl+Z */
void handleSIGTSTP(int sig) {

    (void)sig;

    if (foregroundPid > 0) {

        /* Forward SIGTSTP to foreground process */
        kill(foregroundPid, SIGTSTP);
    }
}

/* Add job to job table */
void addJob(pid_t pid, const char *command) {

    if (jobCount >= MAX_JOBS) {
        printf("Job table is full.\n");
        return;
    }

    jobTable[jobCount].job_id = jobCount + 1;
    jobTable[jobCount].pid = pid;

    strcpy(jobTable[jobCount].command, command);
```

```
    strcpy(jobTable[jobCount].command, command);

    jobTable[jobCount].state = RUNNING;

    jobCount++;
}

/* Display job table */
void displayJobs() {

    printf("\nJob Table\n");
    printf("---------------------------------------\n");
    printf("ID\tPID\tSTATE\t\tCOMMAND\n");
    printf("---------------------------------------\n");

    for (int i = 0; i < jobCount; i++) {

        char *state;

        if (jobTable[i].state == RUNNING)
            state = "RUNNING";

        else if (jobTable[i].state == STOPPED)
            state = "STOPPED";

        else
            state = "COMPLETED";

        printf("[%d]\t%d\t%s\t\t%s\n",
               jobTable[i].job_id,
               jobTable[i].pid,
               state,
               jobTable[i].command);
    }

    printf("---------------------------------------\n");
}

/* Find job */
int findJob(int jobID) {

    for (int i = 0; i < jobCount; i++) {

        if (jobTable[i].job_id == jobID)
            return i;
    }

    return -1;
}

/* Resume stopped job */
```

```c
void resumeJob(int jobID) {

    int index = findJob(jobID);

    if (index == -1) {

        printf("Job [%d] not found.\n", jobID);
        return;
    }

    if (jobTable[index].state != STOPPED) {

        printf("Job [%d] is not stopped.\n", jobID);
        return;
    }

    printf("\nSending SIGCONT to Job [%d]...\n",
           jobID);

    if (kill(jobTable[index].pid, SIGCONT) == -1) {

        perror("SIGCONT");
        return;
    }

    jobTable[index].state = RUNNING;

    printf("Job [%d] resumed successfully.\n",
           jobID);
}

/* Create foreground job */
void createJob() {

    pid_t pid = fork();

    if (pid < 0) {

        perror("fork");
        return;
    }

    /* Child */
    if (pid == 0) {

        /*
         * Child uses default SIGTSTP behavior.
         */
        signal(SIGTSTP, SIG_DFL);
```

```c
         * Child uses default SIGTSTP behavior.
         */
        signal(SIGTSTP, SIG_DFL);

        printf("Foreground job started.\n");
        printf("Child PID: %d\n", getpid());

        for (int i = 1; i <= 20; i++) {

            printf("Job running... %d/20\n", i);

            sleep(1);
        }

        printf("Job completed normally.\n");

        exit(0);
    }

    /* Parent */
    foregroundPid = pid;

    addJob(pid, "foreground-task");

    printf("Job [%d] started with PID %d.\n",
           jobTable[jobCount - 1].job_id,
           pid);
}

/* Wait for job and update state */
void monitorJob(int index) {

    int status;

    pid_t result = waitpid(
        jobTable[index].pid,
        &status,
        WUNTRACED
    );

    if (result == -1) {

        perror("waitpid");
        return;
    }

    if (WIFSTOPPED(status)) {

        jobTable[index].state = STOPPED;

        printf("\nJob [%d] has been stopped.\n",
               jobTable[index].job_id);
```

```
    } else if (WIFEXITED(status) ||
               WIFSIGNALED(status)) {

        jobTable[index].state = COMPLETED;

        printf("\nJob [%d] has completed.\n",
               jobTable[index].job_id);
    }

    foregroundPid = -1;
}

int main() {

    /* Register SIGTSTP handler */
    signal(SIGTSTP, handleSIGTSTP);

    printf("SIGTSTP Job Suspension System\n");
    printf("-----------------------------\n");

    printf("Shell PID: %d\n", getpid());

    printf("\nStarting foreground job...\n");

    createJob();

    sleep(1);

    printf("\nPress Ctrl+Z to suspend the job.\n");

    /*
     * Wait until job either stops or completes.
     */
    monitorJob(0);

    displayJobs();

    /* Resume stopped job */
    if (jobTable[0].state == STOPPED) {

        printf("\nResuming stopped job...\n");

        resumeJob(1);

        displayJobs();

        printf("\nWaiting for resumed job to complete...\n");

        int status;

        waitpid(
```

```
        printf("\nWaiting for resumed job to complete...\n");

        int status;

        waitpid(
            jobTable[0].pid,
            &status,
            0
        );

        jobTable[0].state = COMPLETED;

        printf("\nJob [1] completed after resume.\n");
    }

    printf("\nFinal Job State:\n");

    displayJobs();

    printf("\nSIGTSTP suspension workflow completed successfully.\n");

    return 0;
}
```

Result:

```
sanjana3027@DESKTOP-QJF5AGU:       no Skill20a.c
sanjana3027@DESKTOP-QJF5AGU:       c Skill20a.c -o s20a
sanjana3027@DESKTOP-QJF5AGU:       s20a
SIGTSTP Job Suspension System
------------------------------
Shell PID: 3679

Starting foreground job...
Job [1] started with PID 3680.
Foreground job started.
Child PID: 3680
Job running... 1/20
Job running... 2/20

Press Ctrl+Z to suspend the job.
Job running... 3/20
Job running... 4/20
Job running... 5/20
^Z
Job [1] has been stopped.

Job Table
----------------------------------------
ID      PID     STATE           COMMAND
----------------------------------------
[1]     3680    STOPPED         foreground-task
----------------------------------------

Resuming stopped job...

Sending SIGCONT to Job [1]...
Job [1] resumed successfully.

Job Table
----------------------------------------
ID      PID     STATE           COMMAND
----------------------------------------
[1]     3680    RUNNING         foreground-task
----------------------------------------

Waiting for resumed job to complete...
Job running... 6/20
Job running... 7/20
Job running... 8/20
Job running... 9/20
Job running... 10/20
Job running... 11/20
Job running... 12/20
Job running... 13/20
Job running... 14/20
Job running... 15/20
Job running... 16/20
Job running... 17/20
Job running... 18/20
Job running... 19/20
Job running... 20/20
Job completed normally.

Job [1] completed after resume.

Final Job State:

Job Table
----------------------------------------
ID      PID     STATE           COMMAND
----------------------------------------
[1]     3680    COMPLETED               foreground-task
----------------------------------------

SIGTSTP suspension workflow completed successfully.
sanjana3027@DESKTOP-QJF5AGU:
```

To Create Process Groups, Assign Group IDs, Transfer Terminal Control, Restore Ownership, Coordinate Signals, Validate Behavior.

Code:

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>
#include <errno.h>

int main() {

    pid_t child_pid;
    pid_t child_pgid;
    pid_t shell_pgid;
    int status;

    printf("Process Group and Terminal Control Demo\n");
    printf("---------------------------------------\n");

    /* Get shell/process group information */
    shell_pgid = getpgrp();

    printf("Parent PID      : %d\n", getpid());
    printf("Parent PGID     : %d\n", shell_pgid);
    printf("Terminal PGID   : %d\n",
           tcgetpgrp(STDIN_FILENO));

    /*
     * Ignore terminal-generated job-control signals
     * in the parent while transferring terminal control.
     */
    signal(SIGTTOU, SIG_IGN);
    signal(SIGTTIN, SIG_IGN);
    signal(SIGTSTP, SIG_IGN);

    /* Create child */
    child_pid = fork();

    if (child_pid < 0) {

        perror("fork");
        return 1;
    }

    /* ---------------- CHILD ---------------- */
    if (child_pid == 0) {

        /*
         * Create a new process group.
         * Child becomes the leader of the group.
         */
        if (setpgid(0, 0) == -1) {

            perror("setpgid");
            exit(1);
        }

        child_pgid = getpgrp();

        printf("\nChild Process\n");
        printf("-------------\n");
        printf("Child PID  : %d\n", getpid());
        printf("Child PGID : %d\n", child_pgid);

        /*
         * Wait for terminal control.
         */
        printf("Child waiting for terminal control...\n");

        sleep(2);

        printf("Child executing with terminal ownership.\n");

        sleep(2);

        printf("Child execution completed.\n");

        exit(0);
    }

    /* ---------------- PARENT ---------------- */

    /*
     * Put child into its own process group.
     */
    if (setpgid(child_pid, child_pid) == -1) {

        if (errno != EACCES) {
            perror("setpgid parent");
            return 1;
        }
    }

    child_pgid = child_pid;

    printf("\nChild created successfully.\n");
    printf("Child PID  : %d\n", child_pid);
    printf("Child PGID : %d\n", child_pgid);

    sleep(1);

    /*
     * Transfer terminal ownership to child group.
     */
    printf("\nTransferring terminal control...\n");
```

```
     * Transfer terminal ownership to child group.
     */
    printf("\nTransferring terminal control...\n");

    if (tcsetpgrp(STDIN_FILENO, child_pgid) == -1) {

        perror("tcsetpgrp - transfer");
        return 1;
    }

    printf("Terminal control transferred to PGID %d.\n",
           child_pgid);

    /*
     * Wait for child to finish.
     */
    waitpid(child_pid, &status, 0);

    printf("\nChild process finished.\n");

    /*
     * Restore terminal ownership to parent.
     */
    printf("Restoring terminal control to parent...\n");

    if (tcsetpgrp(STDIN_FILENO, shell_pgid) == -1) {

        perror("tcsetpgrp - restore");
        return 1;
    }

    printf("Terminal control restored to PGID %d.\n",
           shell_pgid);

    /*
     * Verify terminal ownership.
     */
    printf("\nTerminal Ownership Verification\n");
    printf("-------------------------------\n");

    printf("Current Terminal PGID: %d\n",
           tcgetpgrp(STDIN_FILENO));

    if (tcgetpgrp(STDIN_FILENO) == shell_pgid) {

        printf("Terminal ownership restored successfully.\n");

    } else {

        printf("Terminal ownership verification failed.\n");
    }
```

```
    /*
     * Verify terminal ownership.
     */
    printf("\nTerminal Ownership Verification\n");
    printf("-------------------------------\n");

    printf("Current Terminal PGID: %d\n",
           tcgetpgrp(STDIN_FILENO));

    if (tcgetpgrp(STDIN_FILENO) == shell_pgid) {

        printf("Terminal ownership restored successfully.\n");

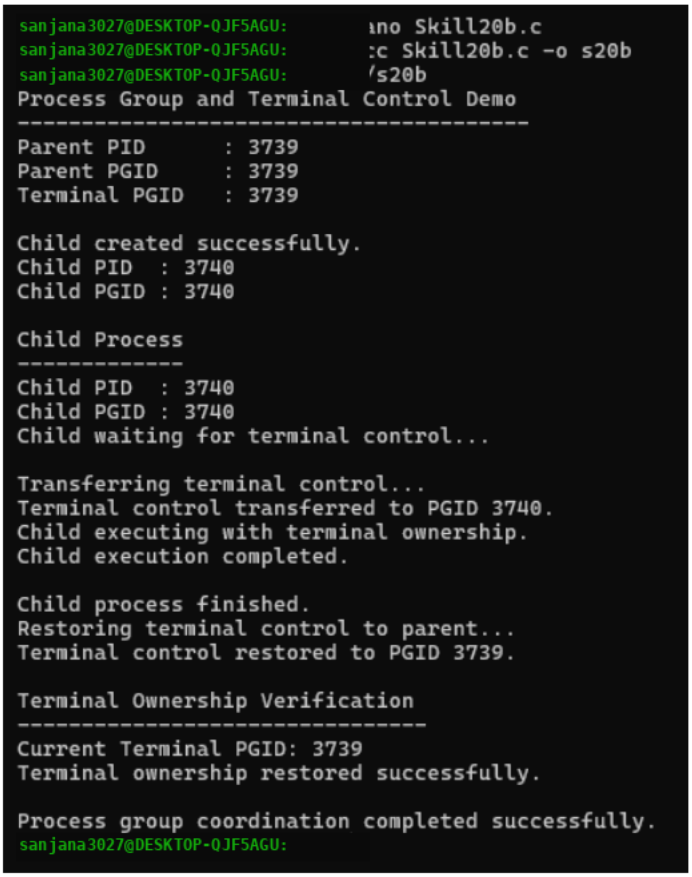
    } else {

        printf("Terminal ownership verification failed.\n");
    }

    printf("\nProcess group coordination completed successfully.\n");

    return 0;
}
```

Result:


sanjana3027@DESKTOP-QJF5AGU: ano Skill20b.c
sanjana3027@DESKTOP-QJF5AGU: :c Skill20b.c -o s20b
sanjana3027@DESKTOP-QJF5AGU: /s20b
Process Group and Terminal Control Demo
Parent PID : 3739
Parent PGID : 3739
Terminal PGID : 3739
Child created successfully.
Child PID : 3740
Child PGID : 3740
Child Process
Child PID : 3740
Child PGID : 3740
Child waiting for terminal control...
Transferring terminal control...
Terminal control transferred to PGID 3740.
Child executing with terminal ownership.
Child execution completed.
Child process finished.
Restoring terminal control to parent...
Terminal control restored to PGID 3739.
Terminal Ownership Verification
Current Terminal PGID: 3739
Terminal ownership restored successfully.
Process group coordination completed successfully.
sanjana3027@DESKTOP-QJF5AGU: